Operating System

Instructor: Dr. Qiyu Liu

Southwest University

March 13, 2025

Table of Contents

Process Abstraction (ver 1.0)

Process Control Block A ver 1.0 implementation

```
struct process_v1 {
    struct context *ctx; // processor ctx
    struct vmspace *vmspace; // user virtual memory space
    void *stack; // kernel stack
};
```

Question:

- Why do we separate user mem space (i.e., user stack + heap) and kernel stack?

Process Creation (ver 1.0)

Objective: initialize PCB and prepare the necessary execution environment for a process.

```
int process_create(char *path, char *argv[], char *envp[]) {
    // create a new PCB
    struct process *new_proc = alloc_process();
    // initialize vmspace
    init_vmspace(new_proc->vmspace);
    new_proc->vmspace->pgtbl = alloc_page();
    // initialize kernel stack
    init_kern_stack(new_proc->stack);

    // load executable file and map to memory space
    ...
    // prepare execution environment
    ...
}
```

Process Creation (ver 1.0) Cont.

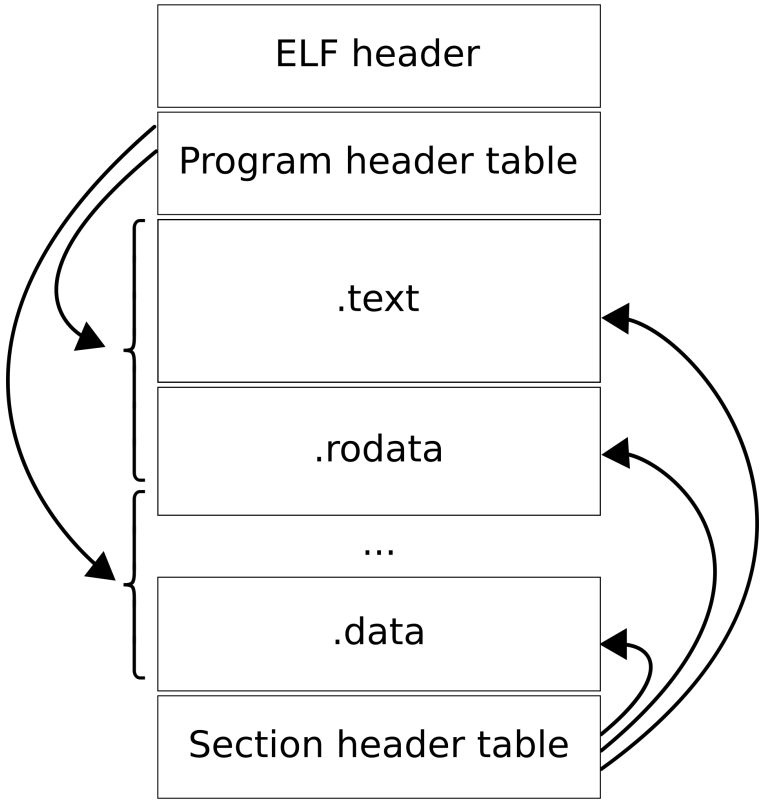
Objective: initialize PCB and prepare the necessary execution environment for a process.

```
int process_create(char *path, char *argv[], char *envp[]) {
    ...
    // prepare execution environment
    // 1. create USER stack
    void *stack = alloc_stack(STACKSIZE);
    vmspace_map(new_proc->vmspace, stack);

    // 2. push arg to stack
    prepare_env(stack, argv, envp);

    // init context
    init_process_ctx(new_proc->ctx);
}
```

Executable and Linkable Format
ELF Layout



Process Creation – C Example

Question: How do a C program start?

```
int main(int argc, char *argv[]) {
    ...
    return 0;
}
```

Answer: glibc helps us with the dirty work!

Hint: main function is invoked by __libc_start_main (check asm)

CPU Context Initialization

- Generic registers
- PC (Program Counter)
- PSTATE (Processor State)
- SP (Stack Pointer)

Process Abstraction (ver 1.0)

Process Control Block A ver 1.0 implementation

```
struct process_v1 {
    struct context *ctx; // processor ctx
    struct vmspace *vmspace; // user virtual memory space
    void *stack; // kernel stack
};
```

Question:

- Why do we separate user mem space (i.e., user stack + heap) and kernel stack?

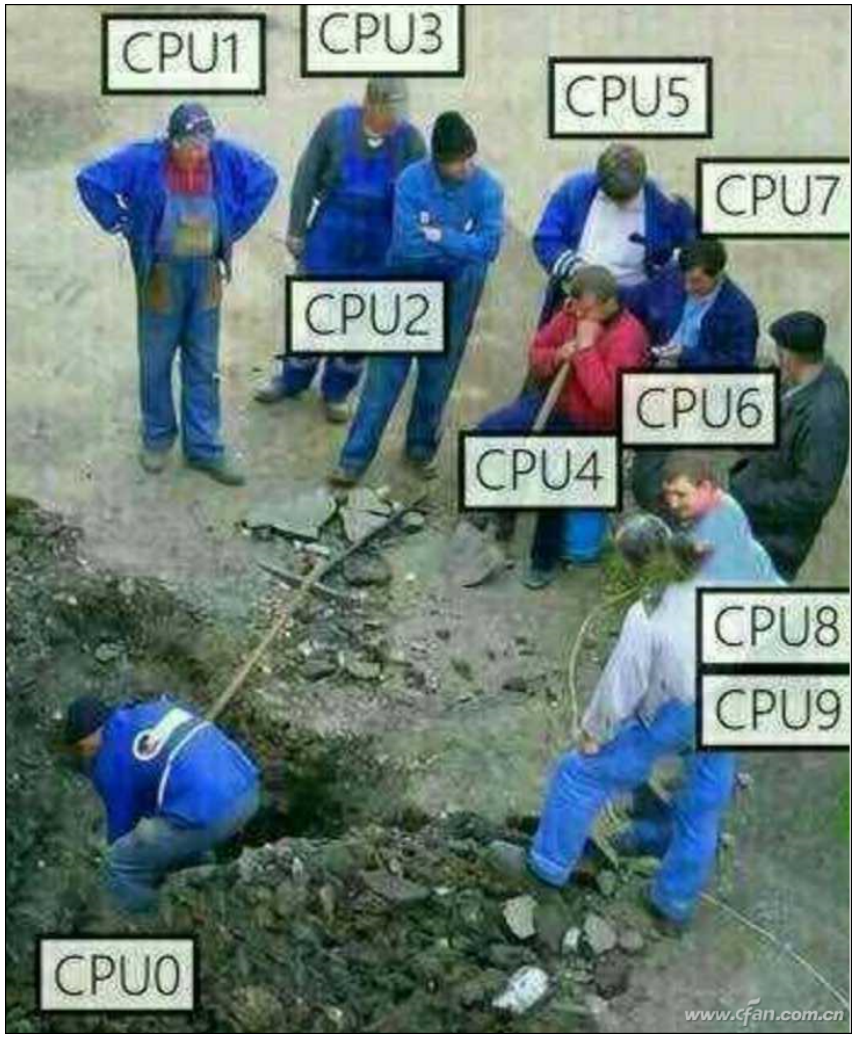
Process Exit (ver 1.0)

Question: What should we do after ending a process?
Answer: Resource recycling!!

```
void process_exit_v1(void) {
    // destroy ctx
    destroy_ctx(curr_proc->ctx);
    // destroy kernel stack
    destroy_kern_stack(curr_proc->stack);
    // destroy vm space
    destroy_vmspace(curr_proc->vmspace);
    // destroy PCB
    destroy_process(curr_proc);
    // schedule and run next process
    schedule();
}
```

Schedule

TLDR: Pick a ready process to run. (details will be learned in the future)



Process Exit (ver 1.0) Cont.

Question: How do a C program exit?

```
int main(int argc, char *argv[]) {
    ...
    return 0;
}
```

Answer: glibc helps us with the dirty work!

Hint: main function is invoked by __libc_start_main (check asm)

Process Waiting (ver 1.0)

Motivation: Processes are working together (collaboration).
Example: Let's call "ls" in shell. When "ls" terminates, we return back to shell, otherwise we are waiting.

```
void process_waitpid_v1(int id) {
    while (1) {
        bool not_exit = FALSE;
        for (auto *proc : all_processes) {
            if (proc->pid == id)
                not_exit = TRUE;
        }
        if (not_exit)
            schedule(); // WHY??
        else
            return; // WHY AGAIN??
    }
}
```

Process Abstraction (ver 2.0)

Process Control Block

A ver 2.0 implementation to support waiting operation.

```
struct process_v2 {
    struct context *ctx; // processor ctx
    struct vmspace *vmspace; // user virtual memory space
    void *stack; // kernel stack
    int pid; // process IDENTIFIER
};
```

Process Exit (ver 2.0)

Limitation of ver 1.0:

- Sometimes we want to return status of an exited process. (WHY?)
- process_waitpid only waits for the termination without knowing the exit status. (return 0 of main function)

```
void process_exit_v1(void) {
    // destroy ctx
    destroy_ctx(curr_proc->ctx);
    // destroy kernel stack (this is problematic!!)
    destroy_kern_stack(curr_proc->stack);
    // destroy vm space
    destroy_vmspace(curr_proc->vmspace);
    // destroy PCB
    destroy_process(curr_proc);
    // schedule and run next process
    schedule();
}
```

Process Exit (ver 2.0)

This is ver 2.0 implementation of exit to support passing status.

```
void process_exit_v2(int status) {
    // destroy ctx
    destroy_ctx(curr_proc->ctx);
    // destroy vm space
    destroy_vmspace(curr_proc->vmspace);
    // save exit status
    curr_proc->exit_status = status;
    // mark process as "exit"
    curr_proc->is_exit = TRUE;
    // schedule and run next process
    schedule();
}
```

Question: Emmm, something missing? (Stack/PCB)

Process Abstraction (ver 3.0)

Process Control Block

A ver 3.0 implementation to support waiting operation.

```
struct process_v3 {
    struct context *ctx; // processor ctx
    struct vmspace *vmspace; // user virtual memory space
    void *stack; // kernel stack
    int pid; // process IDENTIFIER
    int exit_status; // exit status
    bool is_exit; // exit flag
};
```

Process Waiting (ver 2.0)

```
void process_waitpid_v1(int id, int *status) {
    while (1) {
        bool not_exist = TRUE;
        for (auto *proc : all_processes) {
            if (proc->pid == id) {
                not_exist = FALSE;
                if (proc->is_exit) {
                    *status = proc->exit_status;
                    destroy_kern_stack(proc->stack);
                    destroy_process(proc);
                    return;
                } else schedule();
            }
        }
        if (not_exist) return;
    }
}
```

Process Isolation

- process_waitpid can monitor ANY process (given pid) and get the return status.
- Weak isolation! (security issues)
- **Question**: Which process can invoke process_waitpid ?
- **Answer**: Parent process have the control to children processes

Process Abstraction (ver 4.0)

Process Control Block

A ver 4.0 implementation to support waiting operation.

```
struct process_v4 {
    struct context *ctx; // processor ctx
    struct vmspace *vmspace; // user virtual memory space
    void *stack; // kernel stack
    int pid; // process IDENTIFIER
    int exit_status; // exit status
    bool is_exit; // exit flag
    pcb_list *children; // a list of children processes
};
```

Process Waiting (ver 3.0)

```
void process_waitpid_v3(int id, int *status) {
    while (1) {
        bool not_exist = TRUE;
        for (auto *proc : curr_proc->children) {
            if (proc->pid == id) {
                not_exist = FALSE;
                if (proc->is_exit) {
                    *status = proc->exit_status;
                    destroy_kern_stack(proc->stack);
                    destroy_process(proc);
                    return;
                } else schedule();
            }
        }
        if (not_exist) return;
    }
}
```

Some Questions about Resource Recycling

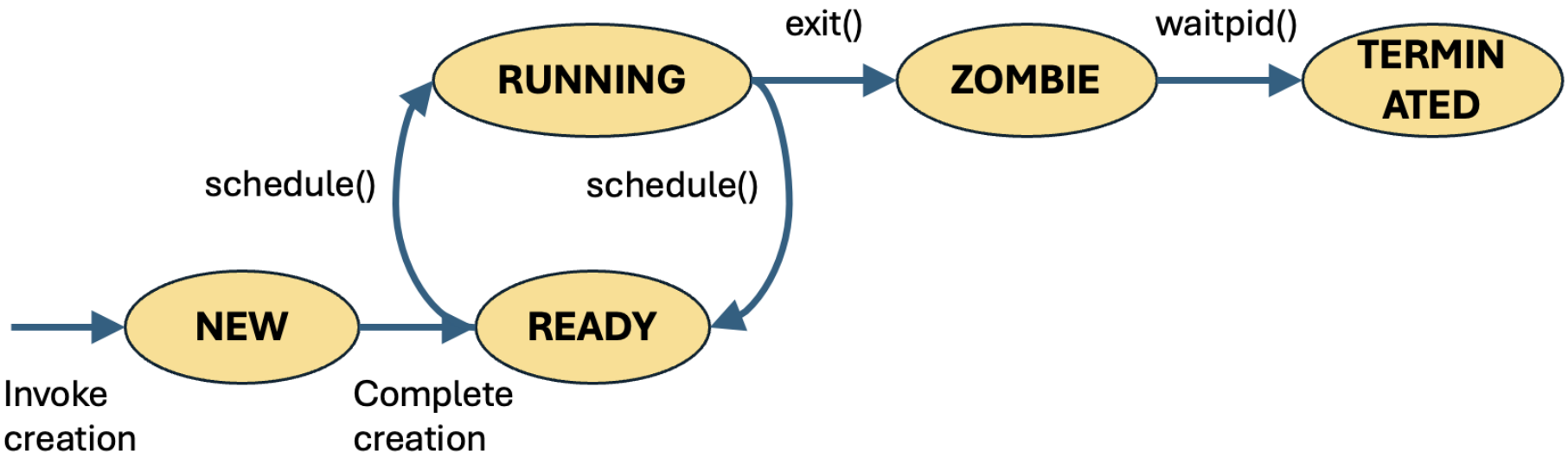
- From the previous discussions, resources allocated to a process (stack, PCB) can be fully recycled only by invoking process_waitpid.
- **Question**: What will happen if no process_waitpid is invoked??
- No worry! A process called init (pid=1) takes over the world!
- Do we have a process with pid=0?

Process Sleep

Motivation: some application requires timing (e.g., clock)

```
void process_sleep(int seconds) {
    struct *date start_time = get_time();
    while (TRUE) {
        struct *date curr_time = get_time();
        if (time_diff(curr_time, start_time() < seconds))
            schedule(); // WHY??
        else
            return; // WHY AGAIN??
    }
}
```

Process Status



Process Abstraction (ver 5.0)

Process Control Block

A ver 5.0 implementation to support waiting operation.

```
enum exec_status {NEW, READY, RUNNING,
                  ZOMBIE, TERMINATED};
struct process_v5 {
    struct context *ctx; // processor ctx
    struct vmspace *vmspace; // user virtual memory space
    void *stack; // kernel stack
    int pid; // process IDENTIFIER
    int exit_status; // exit status
    bool is_exit; // exit flag
    pcb_list *children; // a list of children processes
    enum exec_status exec_status; // process state
};
```

A Simple Scheduler Implementation

```
struct process* pick_next(void) {
    for (struct process* proc : all_processes) {
        if (proc->exec_status == READY) {
            curr_proc->exec_status = READY;
            return proc; // WHY??
        }
    }
}

void schedule() {
    curr_proc = pick_next();
    curr_proc->exec_status = RUNNING;
}
```

Question: Is this a good implementation? How can we improve it?

Q & A